Unit 3 - Run-length algorithms

Essential reading

Sayood, Khalid *Introduction to Data Compression* (Morgan Kaufmann, 1996) [ISBN 1-55860-346-8]. Chapter 6.8.1.

Run-length coding ideas

A run-length algorithm assigns codewords to consecutive recurrent symbols (called runs) instead of coding individual symbols. The main idea is to replace a number of consecutive repeating symbols by a short codeword unit containing three parts: a single symbol, a run-length count and an interpreting indicator.

Example 2.1 *String* KKKKKKKKK*, containing 9 consecutive repeating* K*s, can be replaced by a short unit* r9K *consisting of the symbol* r*,* 9 *and* K*, where* r *represents 'repeating symbol',* 9 *means '9 times of occurrence' and* K *indicates that this should be interpreted as 'symbol* K*' (repeating 9 times).*

Run-length algorithms are very effective if the data source contains many runs of consecutive symbol. The symbols can be characters in a text file, *0*s or *1*s in a binary file or black-and-white pixels in an image.

Although simple, run-length algorithms have been used well in practice. For example, the so-called HDC (Hardware Data Compression) algorithm, used by tape drives connected to IBM computer systems, and a similar algorithm used in the IBM SNA (System Network Architecture) standard for data communications are still in use today.

We briefly introduce the HDC algorithm below.

Hardware data compression (HDC)

In this form of run-length coding, the coder replaces sequences of consecutive identical symbols with three elements:

1. a single symbol
2. a run-length count
3. an indicator that signifies how the symbol and count are to be interpreted.

A simple HDC algorithm

This uses only ASCII codes for:

1. the single symbols, and
2. a total of 123 control characters with a run-length count, including:
   - repeating control characters: $r_2, r_3, \cdots, r_{63}$, and

- non-repeating control characters: $n_1, n_2, \cdots, n_{63}$.

Each r_i, where $i = 2 \cdots 63$, is followed by either another control character or a symbol. If the following symbol is another control character, r_i (alone) signifies i repeating space characters (i.e. blanks). Otherwise, r_i signifies that the symbol immediately after it repeats i times.

Each n_i, where $i = 1 \cdots 63$ is followed by a sequence of i *non-repeating* symbols.

Applying the following 'rules', it is easy to understand the outline of the *encoding* and *decoding* run-length algorithms below:

Encoding

Repeat the following until the end of input file:

Read the source (e.g. the input text) symbols sequentially and

[1] *i.e. a sequence of symbols.*

1. if a string[1] of i ($i = 2 \cdots 63$) consecutive spaces is found, output a single control character r_i
2. if a string of i ($i = 3 \cdots 63$) consecutive symbols other than spaces is found, output two characters: r_i followed by the repeating symbol
3. otherwise, identify a longest string of $i = 1 \cdots 63$ non-repeating symbols, where there is no consecutive sequence of 2 spaces or of 3 other characters, and output the non-repeating control character n_i followed by the string.

Example 2.2 GGGuuuuuuBCDEFGuu55GHJKuLM777777777777

can be compressed to r_3Gr_6n_6BCDEFG$r_2n_9$55LJLMr_{12}7

Solution

1. The first 3 Gs are read and encoded by r_3G.
2. The next 6 spaces are found and encoded by r_6.
3. The non-repeating symbols BCDEFG are found and encoded by n_6BCDEFG.
4. The next 2 spaces are found and encoded by r_2.
5. The next 9 non-repeating symbols are found and encoded by $n_9$55GHJKULM.
6. The next 12 '7's are found and encoded by r_{12}7.

Therefore the encoded output is: r_3Gr_6n_6BCDEFG$r_2n_9$55LJLMr_{12}7.

Decoding

The decoding process is similar to the encoding and can be outlined as follows:

Repeat the following until the end of input coded file:

Read the codeword sequence sequentially and

1. if a r_i is found, then check the next codeword.
   (a) if the codeword is a control character, output i spaces.
   (b) otherwise output i (ASCII codes of) repeating symbols.
2. otherwise, output the next i non-repeating symbols.

Observation

1. It is not difficult to observe from a few examples that the performance of the HDC algorithm (as far as the compression ratio concerns) is:
   - excellent[2] when the data contains many runs of consecutive symbols
   - poor when there are many segments of non-repeating symbols.

   Therefore, run-length algorithms are often used as a subroutine in other more sophisticated coding.

[2] *It can be even better than entropy coding such as Huffman coding.*

2. The decoding process of HDC is simpler than the encoding one[3]
3. The HDC is non-adaptive because the model remains unchanged during the coding process.

[3] *HDC is one of the so-called 'asymmetric' coding methods. Please see page 24 for definition.*

Learning outcomes

On completion of your studies in this chapter, you should be able to:

- state what a Run-length algorithm is
- explain how a Run-length algorithm works
- explain under what conditions a Run-length algorithm may work effectively
- explain, with an example, how the HDC algorithm works.

Activities

1. Apply the HDC (Hardware Data Compression) algorithm to the following sequence of symbols:

   kkkkuuuuuuuuuuuuuguuhh5522777666abbbbcmmjuu##

   Show the compressed output and explain the meaning of each control symbol.

2. Explain how the compressed output from the above question can be reconstructed using the decompressing algorithm.

3. Provide an example of a source file on which the HDC algorithm would perform very badly.

Laboratory

1. Based on the outline of the simple HDC algorithm, derive your version of the HDC algorithm in pseudocode which allows an easy implementation in Java (or C, C++).

2. Implement your version of HDC algorithm in Java. Use "MyHDC" as the name of your main class/program.

3. Provide two source files "good.source" and "bad.source", on which HDC would perform very well and very badly respectively. Indicate your definition of "good" and "bad" performance.

   [Hint] Define the input and the output of your (compression and decompression) algorithms first.

Sample examination questions

1. Describe with an example how a Run-Length Coder works.

2. Apply the HDC (Hardware Data Compression) algorithm to the sequence:

   uuuuuuuBCuuuAu1144330000uuEFGHHHH

   Demonstrate the compressed output and explain the meaning of each control symbol.